



华南师范大学

本科学生实验（实践）报告

院 系： 计算机学院

实验课程： 编译原理

实验项目： 实验1 C++单词拼装分类器

指导老师： 黄煜廉

开课时间： 2023 ~ 2024 年度第二学期

专 业： 计算机科学与技术

班 级： 22级计科2班

学生姓名： 黄兆清

华南师范大学教务处

华南师范大学实验报告

学生姓名 黄兆清 学号 20222131044
专 业 计算机科学与技术 年级、班级 22级计科2班
课程名称 编译原理 实验项目 实验1 C++单词拼装分类器
实验类型 ☐验证 ☐设计 ☐综合 实验时间 2024 年 3 月 8 日
实验指导老师 黄煜廉 实验评分

一、实验内容

1. 把 C++ 源代码中的各类单词（记号）进行拼装分类。

C++ 语言包含了几种类型的单词（记号）：标识符，关键字，数（包括二进制、十进制、八进制和十六进制的整数、浮点数），字符串、注释、特殊符号（分界符）和运算符等【详细的单词类别及拼装规则见详细阅读下面提供的网址中的页面文字说明】。

2. 要求应用程序应为 Windows 界面（窗口式图形界面）：使用对话框打开一个 C++ 源文件，并使用对话框列出所有可以拼装的单词（记号）及其分类。

3. 实验设计实现需要采用的编程语言：C++ 程序设计语言

4. 根据实验的要求组织必要的测试数据（要能对实验中要求的各类单词都能做完备的测试）

5. 书写完善的软件设计文档。

6. C++ 各类单词的拼装约定网址：

(1) <https://learn.microsoft.com/zh-cn/cpp/cpp/cpp/lexical-conventions?view=msvc-170>

(2) <https://learn.microsoft.com/zh-cn/cpp/cpp/cpp/cpp-built-in-operators-precedence-and-associativity?view=msvc-170q>

二、实验目的

1. 熟悉编译原理中对源代码的扫描处理
2. 提高 C++ 编程能力，加深对编程语言的理解和应用。

三、实验文档：

1. 实验项目概述：

本实验项目旨在开发一个基于 Windows 界面的应用程序，通过对话框打开 C++ 源文件，并对源文件中的各类单词（记号）进行分类和拼装。主要包括以下步骤和功能：

(1) 界面设计：

采用窗口式图形界面，包括文件打开对话框和单词分类显示对话框。

(2) 文件操作：

打开文件对话框，允许用户选择要分析的 C++ 源文件。

(3) 单词分类：

读取用户选择的 C++ 源文件，逐行分析源文件中的内容。

依照拼装规则对每行文件中的各类单词进行分类。

将分类结果返回到窗口程序，将中间数据结果转化为文本并显示。

(4) 显示分类结果：

接收分类结果，将结果显示在对话框中。

2. 实验项目设计

(1) 开发框架：使用 Qt 框架来实现窗口界面，提供了丰富的 GUI 组件和功能支持。

(2) 数据结构：使用 `std::vector<std::pair<std::string, TOKEN>>` 作为储存结果的数据结构，其中枚举类型 `TOKEN` 用于表示单词的类型。

(3) 算法设计：实现 `scanner` 函数，用于分析输入的一行数据并将结果写入数据结构。在 `scanner` 中根据特征调用相应的子程序进行分析，子程序遇到问题时退出并返回错误码。

(4) 开发策略：采用测试驱动开发，先为数字处理、标识符和关键字处理、注释处理、字符串处理、特殊符号处理和运算符处理等模块编写对应的测试程序，再编写相应的程序。分析程序测试使用简单 `makefile` 进行管理，在命令行下进行，方便快捷进行测试而无需过多干预源代码；图形化程序测试手动测试即可。

(5) 测试方法：

正确性测试：输入正确的类型，使用 assert 对比识别结果与预期结果。

效率测试：生成一个规模较大的测试 C++ 源程序，测试程序稳定性及处理速度。

3. 数据结构选择

文件每行存储类型：std::string

单词类型：enum TOKEN

错误类型：enum ERROR

单个识别结果：std::pair<std::string, TOKEN>

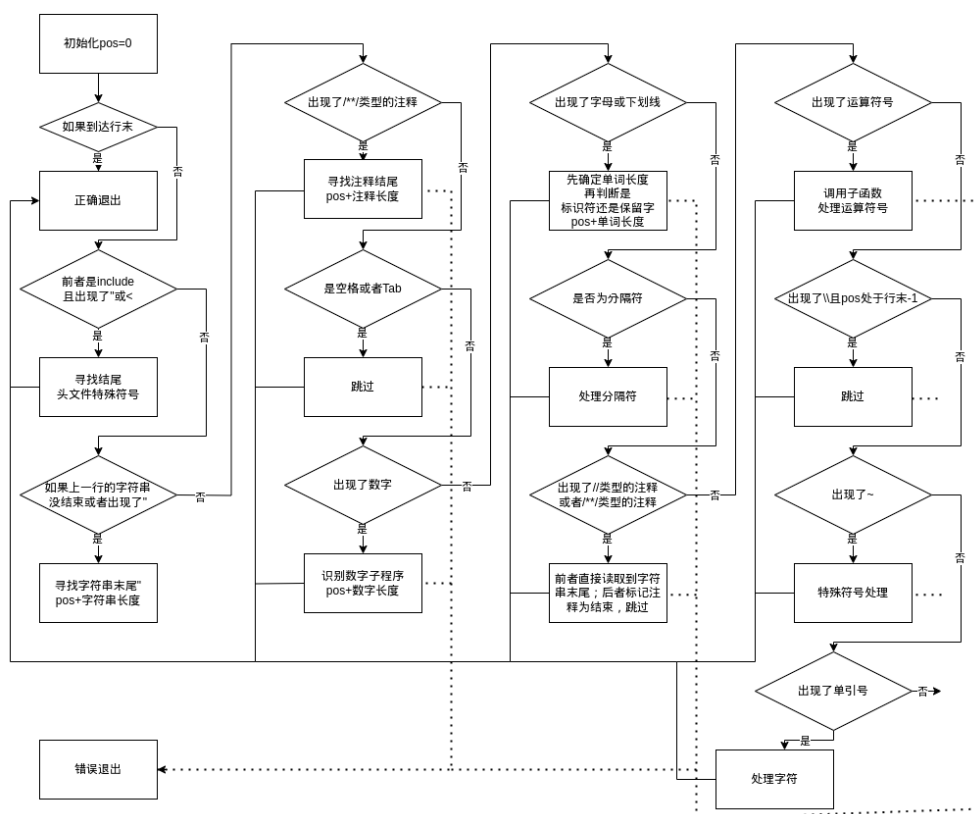
所有识别结果：std::vector<std::pair<std::string, TOKEN>>

输出文本格式：QString

4. 关键算法设计

(1) 识别主程序：

采用读取前几个字符确定进入子程序或是直接处理，程序流程图如下：

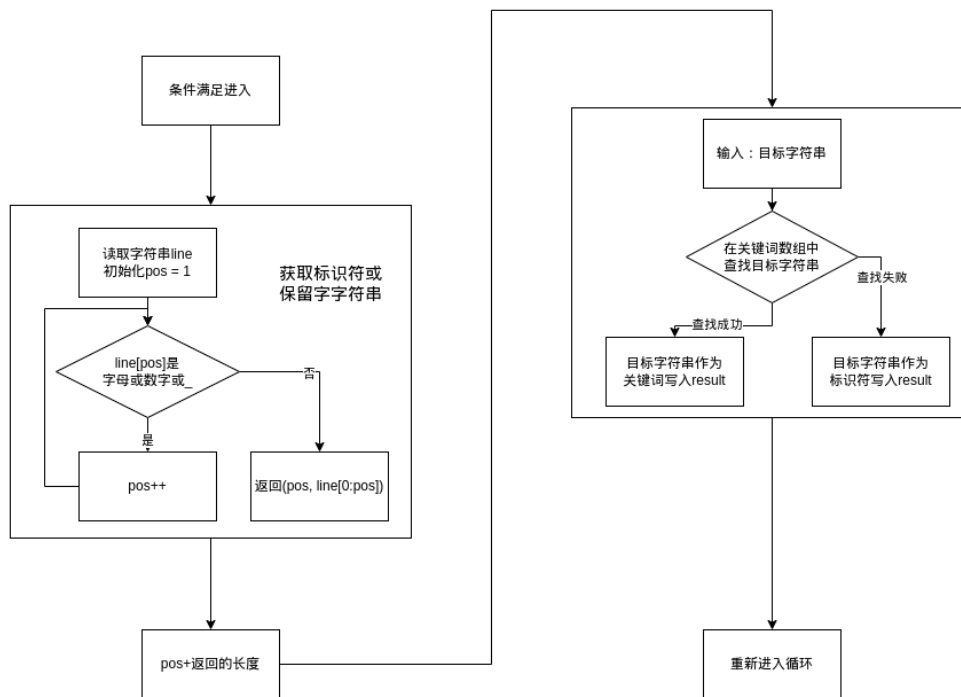


(2) 子模块设计思路：

以标识符_保留字检测为例，先在循环中识别特征，再进入相应的处理子模块，得

到结果之后，将结果添加进结果数组，将读取位置移动字符串长度。

流程图如下：



其他子模块的设计思路基本一致，下面简述一些子模块函数的作用：

```
大纲
word(string, vector<pair<string,TOKEN>>&)
isKeyword(string)
isDelimiter(string)
isOperator(char)
Operator(string, vector<pair<string,TOKEN>>&)
number(string, vector<pair<string,TOKEN>>&)
```

word：在上图的流程图中有所体现。

isDelimiter：对给定的字符串，检测是否为分隔符。

isOperator：对给定的字符，检测是否为单字符符号，由于所有双字符符号的首字符是单字符符号，所以可以作为检测是否为符号。

Operator：处理操作符号。对给定的字符串，取前两个作为子字符串，与所有双字符符号进行比对，如果成功匹配，作为双字符符号添加进结果，如果不成功匹配，则

取首字符作为单字符符号添加进结果。最后返回符号长度。

number：处理数字。开头是数字进入这一个处理子模块。

连续读取字符，读取到 x 或 X 检查是否为十六进制数，并标记；

当读取到.时，如果前面已经有.或者是 16 进制数或者开头为 0 且小数点位置不在第二位（即八进制数），则错误退出；

如果读取到 e,检查是否为十六进制数或是八进制数或是已经出现了 e；

如果读取到小数点，检查是否出现过 e、是否为十六进制数、是否为 8 进制数；

读取到 8 和 9 时，检查是否为八进制数；

读取到 a-f 时，检查是否为十六进制数；

读取到空白、运算符号、分隔符就截止。

分整数和浮点数添加进结果，并返回数字长度。

(3)测试内容：

子模块测试：在不同的测试文件中包含代表性测试数据，读取每个文件，使用断言检测读出每个元素的类型与目标类型是否相同。

主要测试代码：

```
void testFile(string filename, TOKEN test) {
    vector<string> lines = readFile(filename);
    vector<pair<string, TOKEN>> result;
    for (string i : lines) {
        assert(readLine(i, result) == NoError);
    }
    for (auto i : result) {
        assert(i.second == test);
    }
}
```

```
void testLongFile(string filename) {
    vector<string> lines = readFile(filename);
    vector<pair<string, TOKEN>> *result = new vector<pair<string, TOKEN>>();
    for (string i : lines) {
        for (int j = 0; j < 1e5; j++) {
            assert(readLine(i, *result) == NoError);
        }
    }
    assert(result->size() == lines.size() * 1e5);
    delete result;
}
```

一些测试数据如下图：

The screenshot displays a C++ development environment with several test data files open in the top pane and a code editor in the bottom pane. The files include `testCharData.txt`, `testCommentData.txt`, `testDelimiterData.txt`, `testdata.txt`, `testFloatData.txt`, `testIdentifierData.txt`, `testIntegerData.txt`, `testKeywordData.txt`, `testOperatorData.txt`, `testSpecial_symbolData.txt`, and `testStringData.txt`. The code editor shows a function for tokenizing text, with line numbers 155 to 295 visible. The function iterates through a string, identifying tokens based on whitespace and special symbols, and stores them in a vector.

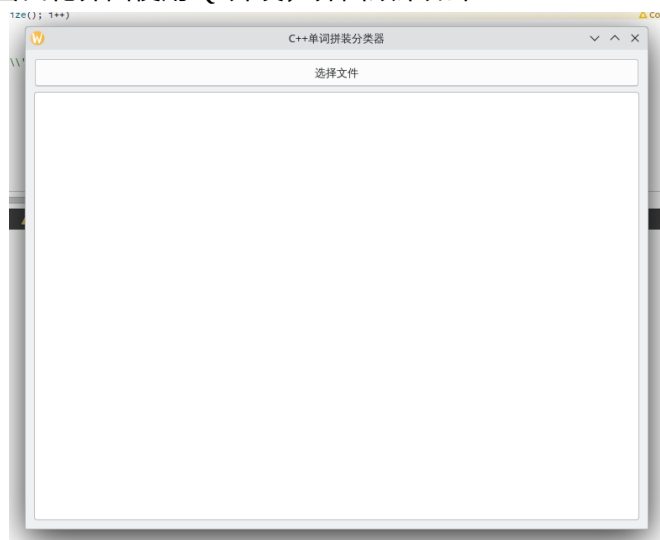
整体文本测试：读入一个综合性程序文本进行相应的测试，目测结果是否成立。

效率测试：对测试的综合性程序文本重复 $1e4$ 次，检测读取长文本时的稳定性。

图形界面测试：手动选择测试程序，观测是否显示正常。

(4) 图形化界面设计：

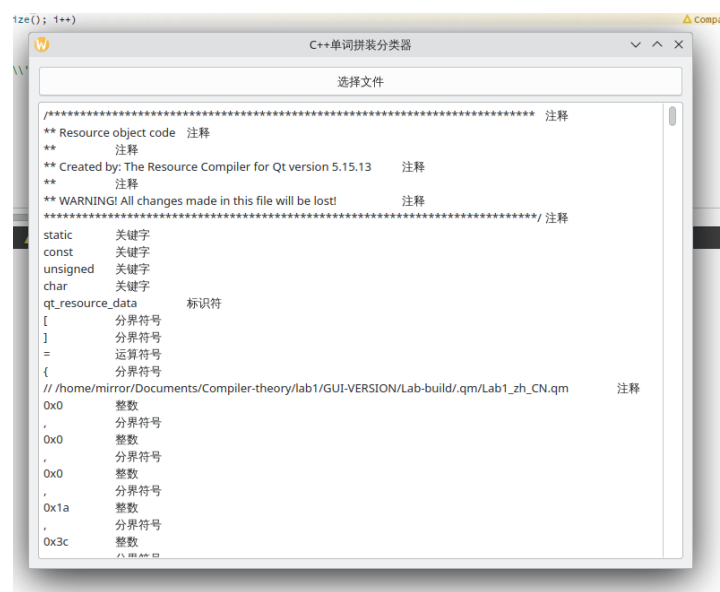
窗口化界面使用 Qt 开发，界面效果如下：



主函数监听按钮点击事件。按钮一旦被点击，相应的槽函数就调用系统的文件资源管理器，等待用户选择文件路径。路径选择结束，调用 `readFile` 对文件进行读取

分析,再通过转换函数,把 token 转化为 QString 显示在文本框内,最后刷新文本框即完成显示。

显示效果如图：



四、实验总结（心得体会）

通过此次实验，我对词法分析拼转器有了更深入的了解，对错误检查也有了初步的认知。在程序的编写过程中，发现了很多自己以前忽略掉的 C++语法，对 C++有了更深入的了解。

五、参考文献：

[1] Kenneth C.Louden.编译原理及实践[M].机械工业出版社,2000-3-1.

[2] Microsoft 2024. 词法约定 [EB/OL].[2023/10/18].<https://learn.microsoft.com/zh-cn/cpp/cpp/lexical-conventions?view=msvc-170>.

[3] Microsoft 2024.C++ 内置运算符、优先级和关联性 [EB/OL].[2023/10/15].<https://learn.microsoft.com/zh-cn/cpp/cpp/cpp-built-in-operators-precedence-and-associativity?view=msvc-170>.

[4] 陆文周.Qt5 开发及实例（第 2 版）[M].第 2 版.电子工业出版社,2015-5-1.